main

March 7, 2026

1 Question 1:

[36]:
```
import os
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras import layers, models
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification,␣
↪Trainer, TrainingArguments
import warnings
warnings.filterwarnings('ignore') # Suppress convergence warnings for clean␣
↪output
```

1.1 1a. Load and clean the dataset

[5]:
```
try:
    df = pd.read_excel('data/online_retail.xlsx')

    print("--- Dataset Info ---")
    df.info()

    print("\n--- First 5 Rows ---")
    display(df.head(5))

except FileNotFoundError:
    print("Error: File not found. Please verify the exact name of the file in␣
 ↪the 'data' folder (e.g., Online_Retail.xlsx).")
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   InvoiceNo    541909 non-null  object
 1   StockCode    541909 non-null  object
 2   Description  540455 non-null  object
 3   Quantity     541909 non-null  int64
 4   InvoiceDate  541909 non-null  datetime64[ns]
 5   UnitPrice    541909 non-null  float64
 6   CustomerID   406829 non-null  float64
 7   Country      541909 non-null  object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 33.1+ MB

--- First 5 Rows ---

  InvoiceNo StockCode                          Description  Quantity  \
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
1    536365     71053                  WHITE METAL LANTERN         6
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6

          InvoiceDate  UnitPrice  CustomerID         Country
0 2010-12-01 08:26:00       2.55     17850.0  United Kingdom
1 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
2 2010-12-01 08:26:00       2.75     17850.0  United Kingdom
3 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
4 2010-12-01 08:26:00       3.39     17850.0  United Kingdom
```

[6]:
```
# 1. Handle missing values
df.dropna(inplace=True)

# 2. Remove cancelled transactions (InvoiceNo starts with 'C')
df['InvoiceNo'] = df['InvoiceNo'].astype(str)
df = df[~df['InvoiceNo'].str.startswith('C')]

# 3. Drop duplicates
df.drop_duplicates(inplace=True)

# 4. Standard clean-up: Remove negative quantities/prices (adjustments/returns␣
↪without 'C')
df = df[(df['Quantity'] > 0) & (df['UnitPrice'] > 0)]

# Feature Engineering for Analysis
df['Revenue'] = df['Quantity'] * df['UnitPrice']
# Extract Month-Year for the trend analysis (converting back to timestamp for␣
↪clean plotting)
df['Month_Year'] = df['InvoiceDate'].dt.to_period('M').dt.to_timestamp()

print("Data cleaning complete. Proceeding to analysis...\n")
```

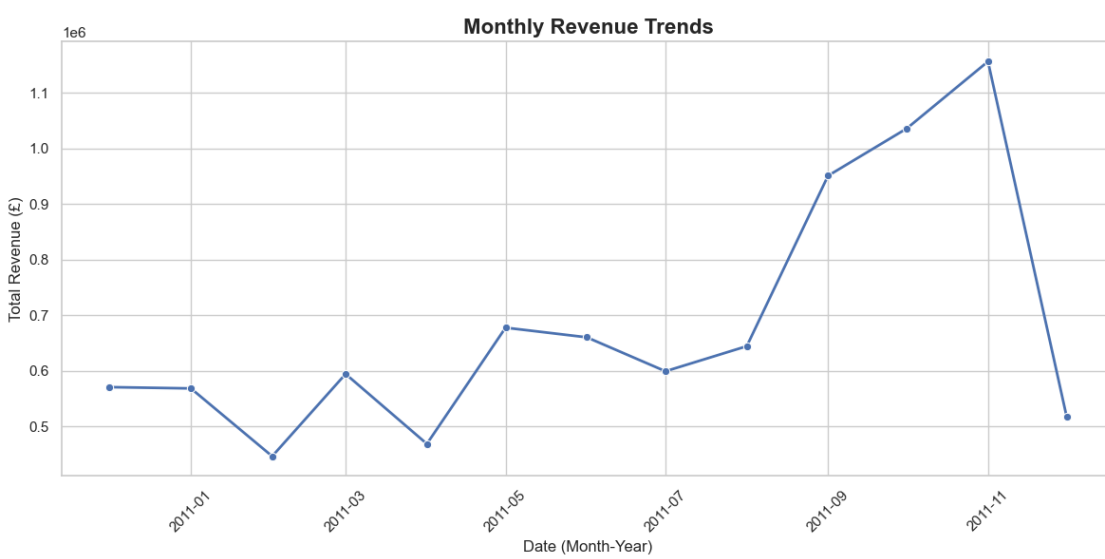
```
Data cleaning complete. Proceeding to analysis…
```

1.2 1b. Descriptive statistics and visualizations

[7]:
```
print("--- Descriptive Statistics ---")
display(df[['Quantity', 'UnitPrice', 'Revenue']].describe().round(2))

# Setup visualization style
sns.set_theme(style="whitegrid")

# Plot 1: Monthly Revenue Trends
plt.figure(figsize=(12, 6))
monthly_revenue = df.groupby('Month_Year')['Revenue'].sum().reset_index()
sns.lineplot(data=monthly_revenue, x='Month_Year', y='Revenue', marker='o',␣
↪color='b', linewidth=2)
plt.xticks(rotation=45)
plt.title('Monthly Revenue Trends', fontsize=16, fontweight='bold')
plt.xlabel('Date (Month-Year)', fontsize=12)
plt.ylabel('Total Revenue (£)', fontsize=12)
plt.tight_layout()
plt.show()

# Plot 2: Top 10 Countries by Total Sales
plt.figure(figsize=(12, 6))
```

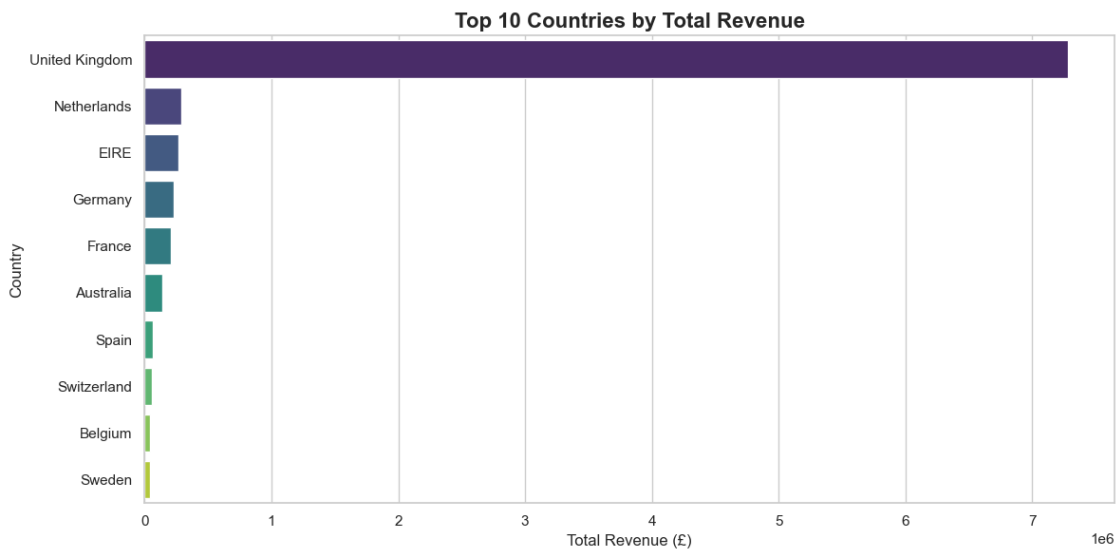
```
top_countries = df.groupby('Country')['Revenue'].sum().
 ↪sort_values(ascending=False).head(10).reset_index()
sns.barplot(data=top_countries, x='Revenue', y='Country', palette='viridis')
plt.title('Top 10 Countries by Total Revenue', fontsize=16, fontweight='bold')
plt.xlabel('Total Revenue (£)', fontsize=12)
plt.ylabel('Country', fontsize=12)
plt.tight_layout()
plt.show()

# Plot 3: Distribution of Order Quantities
# Note: E-commerce data often has massive outliers.
# We filter quantities <= 50 to make the distribution shape visible.
plt.figure(figsize=(10, 6))
sns.histplot(df[df['Quantity'] <= 50]['Quantity'], bins=50, color='coral')
plt.title('Distribution of Order Quantities (Filtered <= 50)', fontsize=16,␣
 ↪fontweight='bold')
plt.xlabel('Quantity per Item', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.tight_layout()
plt.show()
```

```
--- Descriptive Statistics ---

        Quantity  UnitPrice    Revenue
count  392692.00  392692.00  392692.00
mean       13.12       3.13      22.63
std       180.49      22.24     311.10
min         1.00       0.00       0.00
25%         2.00       1.25       4.95
50%         6.00       1.95      12.45
75%        12.00       3.75      19.80
max     80995.00    8142.75  168469.60
```



```
C:\Users\Yahya\AppData\Local\Temp\ipykernel_12904\1613102486.py:21:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.
```

```
sns.barplot(data=top_countries, x='Revenue', y='Country', palette='viridis')
```



2 Question 2

[8]:
```
try:
    df_churn = pd.read_csv('data/Telco_Customer_Churn.csv')

    print("--- Dataset Info ---")
    df_churn.info()

    print("\n--- First 5 Rows ---")
    display(df_churn.head())

except FileNotFoundError:
    print("Error: File not found. Please verify the exact name of the file in␣
 ↪the 'data' folder (e.g., Telco_Customer_Churn.csv).")
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
```

```
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

--- First 5 Rows ---

   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
```

```
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

2.1 2a. Load and Pre-process the Data

[10]:
```
print("Loading and pre-processing data...")
df = pd.read_csv('data/Telco_Customer_Churn.csv')

# 1. Drop customerID (It has no predictive power)
df = df.drop('customerID', axis=1)

# 2. Fix TotalCharges: Convert to numeric, force errors to NaN, fill NaNs with␣
↪0
# (NaNs represent brand new customers with 0 tenure)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(0)

# 3. Encode the Target Variable (Churn: Yes -> 1, No -> 0)
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# 4. One-Hot Encode categorical features
# drop_first=True prevents multicollinearity (important for Logistic Regression)
df_encoded = pd.get_dummies(df, drop_first=True)

# 5. Separate features (X) and target (y)
X = df_encoded.drop('Churn', axis=1)
y = df_encoded['Churn']

# 6. Train/Test Split (80% training, 20% testing, stratified to maintain Churn␣
↪ratio)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42, stratify=y)

# 7. Scale Numerical Features (Important for Logistic Regression and Naive␣
↪Bayes)
scaler = StandardScaler()
num_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

```
Loading and pre-processing data…
```

2.2 2b. Build Models and Compare

[11]:
```
print("Training models and calculating metrics...\n")

# Initialize models
models = {
    'Random Forest': RandomForestClassifier(random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Naive Bayes': GaussianNB()
}

results = []

for name, model in models.items():
    # Train the model
    model.fit(X_train, y_train)

    # Make predictions
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1] # Get probabilities for AUC-ROC

    # Calculate metrics
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)

    # Store results
    results.append({
        'Model': name,
        'Accuracy': round(acc, 4),
        'F1-Score': round(f1, 4),
        'AUC-ROC': round(auc, 4)
    })

# Display Results as a DataFrame (Table)
results_df = pd.DataFrame(results)
print("--- Model Comparison Results ---")
display(results_df)
```

```
Training models and calculating metrics…

--- Model Comparison Results ---
```

	Model	Accuracy	F1-Score	AUC-ROC
0	Random Forest	0.7850	0.5498	0.8248
1	Logistic Regression	0.8062	0.6049	0.8422
2	Naive Bayes	0.6558	0.5719	0.8092

3 Question 3

[12]:
```
# Load the CSV dataset
try:
    df_housing = pd.read_csv('data/AmesHousing.csv')

    print("--- Dataset Info ---")
    # Setting max_cols to ensure it prints all columns instead of truncating
    df_housing.info(max_cols=100)

    print("\n--- First 5 Rows ---")
    display(df_housing.head())

except FileNotFoundError:
    print("Error: File not found. Please verify the exact name of the file in␣
 ↪the 'data' folder (e.g., AmesHousing.csv).")
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2930 entries, 0 to 2929
Data columns (total 82 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Order            2930 non-null   int64
 1   PID              2930 non-null   int64
 2   MS SubClass      2930 non-null   int64
 3   MS Zoning        2930 non-null   object
 4   Lot Frontage     2440 non-null   float64
 5   Lot Area         2930 non-null   int64
 6   Street           2930 non-null   object
 7   Alley            198 non-null    object
 8   Lot Shape        2930 non-null   object
 9   Land Contour     2930 non-null   object
 10  Utilities        2930 non-null   object
 11  Lot Config       2930 non-null   object
 12  Land Slope       2930 non-null   object
 13  Neighborhood     2930 non-null   object
 14  Condition 1      2930 non-null   object
 15  Condition 2      2930 non-null   object
 16  Bldg Type        2930 non-null   object
 17  House Style      2930 non-null   object
 18  Overall Qual     2930 non-null   int64
 19  Overall Cond     2930 non-null   int64
 20  Year Built       2930 non-null   int64
 21  Year Remod/Add   2930 non-null   int64
 22  Roof Style       2930 non-null   object
 23  Roof Matl        2930 non-null   object
 24  Exterior 1st     2930 non-null   object
```

```
25  Exterior 2nd     2930 non-null   object
26  Mas Vnr Type     1155 non-null   object
27  Mas Vnr Area     2907 non-null   float64
28  Exter Qual       2930 non-null   object
29  Exter Cond       2930 non-null   object
30  Foundation       2930 non-null   object
31  Bsmt Qual        2850 non-null   object
32  Bsmt Cond        2850 non-null   object
33  Bsmt Exposure    2847 non-null   object
34  BsmtFin Type 1   2850 non-null   object
35  BsmtFin SF 1     2929 non-null   float64
36  BsmtFin Type 2   2849 non-null   object
37  BsmtFin SF 2     2929 non-null   float64
38  Bsmt Unf SF      2929 non-null   float64
39  Total Bsmt SF    2929 non-null   float64
40  Heating          2930 non-null   object
41  Heating QC       2930 non-null   object
42  Central Air      2930 non-null   object
43  Electrical       2929 non-null   object
44  1st Flr SF       2930 non-null   int64
45  2nd Flr SF       2930 non-null   int64
46  Low Qual Fin SF  2930 non-null   int64
47  Gr Liv Area      2930 non-null   int64
48  Bsmt Full Bath   2928 non-null   float64
49  Bsmt Half Bath   2928 non-null   float64
50  Full Bath        2930 non-null   int64
51  Half Bath        2930 non-null   int64
52  Bedroom AbvGr    2930 non-null   int64
53  Kitchen AbvGr    2930 non-null   int64
54  Kitchen Qual     2930 non-null   object
55  TotRms AbvGrd    2930 non-null   int64
56  Functional       2930 non-null   object
57  Fireplaces       2930 non-null   int64
58  Fireplace Qu     1508 non-null   object
59  Garage Type      2773 non-null   object
60  Garage Yr Blt    2771 non-null   float64
61  Garage Finish    2771 non-null   object
62  Garage Cars      2929 non-null   float64
63  Garage Area      2929 non-null   float64
64  Garage Qual      2771 non-null   object
65  Garage Cond      2771 non-null   object
66  Paved Drive      2930 non-null   object
67  Wood Deck SF     2930 non-null   int64
68  Open Porch SF    2930 non-null   int64
69  Enclosed Porch   2930 non-null   int64
70  3Ssn Porch       2930 non-null   int64
71  Screen Porch     2930 non-null   int64
72  Pool Area        2930 non-null   int64
```

```
 73  Pool QC          13 non-null     object
 74  Fence            572 non-null    object
 75  Misc Feature     106 non-null    object
 76  Misc Val         2930 non-null   int64
 77  Mo Sold          2930 non-null   int64
 78  Yr Sold          2930 non-null   int64
 79  Sale Type        2930 non-null   object
 80  Sale Condition   2930 non-null   object
 81  SalePrice        2930 non-null   int64
dtypes: float64(11), int64(28), object(43)
memory usage: 1.8+ MB

--- First 5 Rows ---

   Order        PID  MS SubClass MS Zoning  Lot Frontage  Lot Area Street  \
0      1  526301100           20        RL         141.0     31770   Pave
1      2  526350040           20        RH          80.0     11622   Pave
2      3  526351010           20        RL          81.0     14267   Pave
3      4  526353030           20        RL          93.0     11160   Pave
4      5  527105010           60        RL          74.0     13830   Pave

  Alley Lot Shape Land Contour  … Pool Area Pool QC  Fence Misc Feature  \
0   NaN       IR1          Lvl  …         0     NaN    NaN          NaN
1   NaN       Reg          Lvl  …         0     NaN  MnPrv          NaN
2   NaN       IR1          Lvl  …         0     NaN    NaN         Gar2
3   NaN       Reg          Lvl  …         0     NaN    NaN          NaN
4   NaN       IR1          Lvl  …         0     NaN  MnPrv          NaN

  Misc Val Mo Sold Yr Sold Sale Type  Sale Condition  SalePrice
0        0       5    2010       WD           Normal     215000
1        0       6    2010       WD           Normal     105000
2    12500       6    2010       WD           Normal     172000
3        0       4    2010       WD           Normal     244000
4        0       3    2010       WD           Normal     189900

[5 rows x 82 columns]
```

3.1 3a. Load and Pre-process the Data

[14]:
```python
print("Loading and pre-processing data...")
df = pd.read_csv('data/AmesHousing.csv')

# 1. Drop identifier columns
df = df.drop(['Order', 'PID'], axis=1)

# 2. Separate Features (X) and Target (y)
X = df.drop('SalePrice', axis=1)
y = df['SalePrice']
```

```
# 3. Handle Missing Values
# Numerical: Fill with median (robust to outliers)
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
X[num_cols] = X[num_cols].fillna(X[num_cols].median())

# Categorical: Fill with 'None' (NaN mostly means the feature is absent, e.g.,␣
↪no garage)
cat_cols = X.select_dtypes(include=['object']).columns
X[cat_cols] = X[cat_cols].fillna('None')

# 4. One-Hot Encode categorical variables
X_encoded = pd.get_dummies(X, drop_first=True)

# 5. Train/Test Split (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.
↪2, random_state=42)

# 6. Scale Features (CRITICAL for regularization)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Total features after One-Hot Encoding: {X_train_scaled.shape[1]}\n")
```

```
Loading and pre-processing data…
Total features after One-Hot Encoding: 276
```

3.2 3b. Build Models and Compare

[15]:
```
print("Training models and calculating metrics...\n")

# Initialize models
# (using slightly adjusted alphas/max_iter to ensure convergence with so many␣
↪features)
models = {
    'Linear Regression': LinearRegression(),
    'Ridge (L2)': Ridge(alpha=10.0),
    'Lasso (L1)': Lasso(alpha=100.0, max_iter=10000),
    'ElasticNet (L1+L2)': ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
}

results = []

for name, model in models.items():
    # Train
```

```
    model.fit(X_train_scaled, y_train)

    # Predict
    y_pred = model.predict(X_test_scaled)

    # Metrics
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    results.append({
        'Model': name,
        'R² (Test)': round(r2, 4),
        'RMSE (Test)': round(rmse, 2)
    })

# Display Results Table
results_df = pd.DataFrame(results)
print("--- Model Comparison Results ---")
display(results_df)
```

```
Training models and calculating metrics…

--- Model Comparison Results ---
                Model  R² (Test)  RMSE (Test)
0   Linear Regression     0.8455     35193.69
1          Ridge (L2)     0.8925     29351.92
2          Lasso (L1)     0.8929     29304.58
3  ElasticNet (L1+L2)     0.8929     29304.33
```

4 Question 4

[16]:
```
# Load the CSV dataset
try:
    df_mall = pd.read_csv('data/mall_customers.csv')

    print("--- Dataset Info ---")
    df_mall.info()

    print("\n--- First 5 Rows ---")
    display(df_mall.head())

except FileNotFoundError:
    print("Error: File not found. Please verify the exact name of the file in␣
 ↪the 'data' folder (e.g., mall_customers.csv).")
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

--- First 5 Rows ---
```

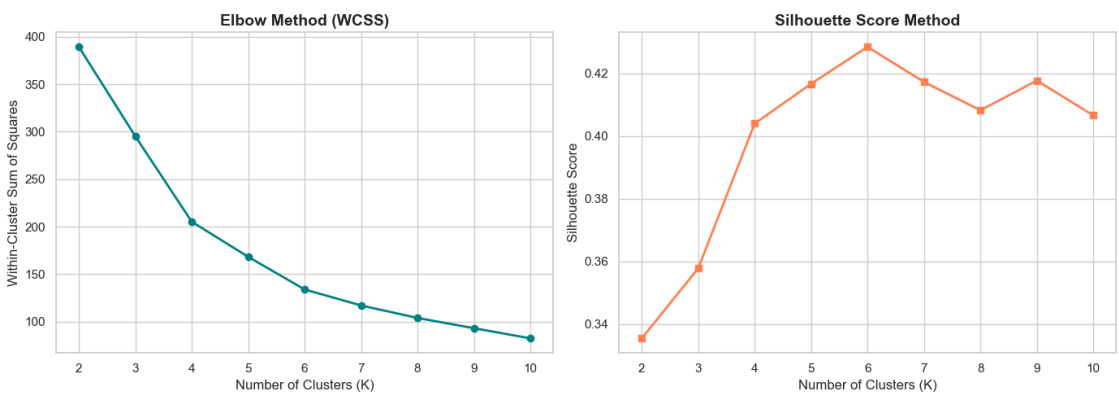
	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

4.1 4a. Data Prep and Finding Optimal K

[18]:
```python
print("Loading and scaling data...")
df = pd.read_csv('data/mall_customers.csv')

# Select the three features
features = ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
X = df[features]

# Scale the data (Critical for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Calculate WCSS (for Elbow) and Silhouette Scores
wcss = []
sil_scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_scaled, kmeans.labels_))

# Plot Elbow and Silhouette Methods
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
sns.set_theme(style="whitegrid")
```

```
# Elbow Plot
ax[0].plot(k_range, wcss, marker='o', color='teal', linewidth=2)
ax[0].set_title('Elbow Method (WCSS)', fontsize=14, fontweight='bold')
ax[0].set_xlabel('Number of Clusters (K)', fontsize=12)
ax[0].set_ylabel('Within-Cluster Sum of Squares', fontsize=12)

# Silhouette Plot
ax[1].plot(k_range, sil_scores, marker='s', color='coral', linewidth=2)
ax[1].set_title('Silhouette Score Method', fontsize=14, fontweight='bold')
ax[1].set_xlabel('Number of Clusters (K)', fontsize=12)
ax[1].set_ylabel('Silhouette Score', fontsize=12)

plt.tight_layout()
plt.show()

# Automatically select the optimal K based on the highest Silhouette Score
optimal_k = k_range[np.argmax(sil_scores)]
print(f"\nOptimal number of clusters based on highest Silhouette Score:␣
↪{optimal_k}")
```

```
Loading and scaling data…
```




```
Optimal number of clusters based on highest Silhouette Score: 6
```

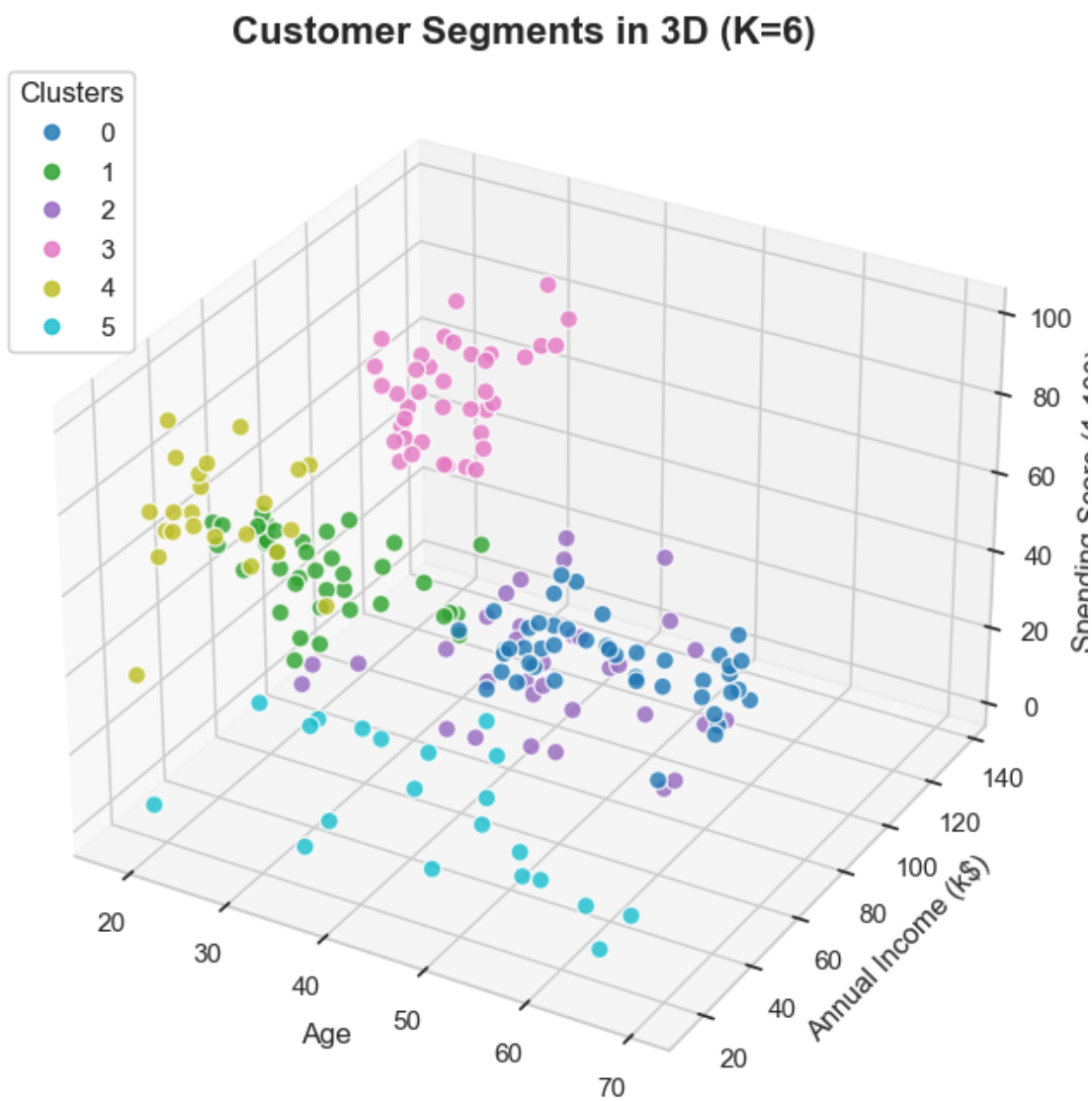
4.2 4b. Final Clustering and 3D Visualization

[19]:
```
print(f"Applying K-Means with K={optimal_k}...")
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = kmeans_final.fit_predict(X_scaled)

# Create a cluster profile to help us name the segments!
cluster_profiles = df.groupby('Cluster')[features].mean().round(1)
```

```python
print("\n--- Cluster Profiles (Averages per Segment) ---")
display(cluster_profiles)

# 3D Scatter Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

# Scatter plot using original (unscaled) data for interpretable axes
scatter = ax.scatter(df['Age'],
                     df['Annual Income (k$)'],
                     df['Spending Score (1-100)'],
                     c=df['Cluster'], cmap='tab10', s=60, alpha=0.8,␣
 ↪edgecolors='w')

ax.set_title(f'Customer Segments in 3D (K={optimal_k})', fontsize=16,␣
 ↪fontweight='bold')
ax.set_xlabel('Age', fontsize=12)
ax.set_ylabel('Annual Income (k$)', fontsize=12)
ax.set_zlabel('Spending Score (1-100)', fontsize=12)

# Add a color bar
legend = ax.legend(*scatter.legend_elements(), title="Clusters", loc="upper␣
 ↪left")
ax.add_artist(legend)

plt.show()
```

```
Applying K-Means with K=6…

--- Cluster Profiles (Averages per Segment) ---
```

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	56.3	54.3	49.1
1	26.8	57.1	48.1
2	41.9	88.9	17.0
3	32.7	86.5	82.1
4	25.0	25.3	77.6
5	45.5	26.3	19.4



5 Question 5

[20]:
```
# Load the Excel dataset
try:
    df_energy = pd.read_excel('data/energy_efficiency.xlsx')

    print("--- Dataset Info ---")
    df_energy.info()

    print("\n--- First 5 Rows ---")
    display(df_energy.head())
```

```
except FileNotFoundError:
    print("Error: File not found. Please verify the exact name of the file in␣
 ↪the 'data' folder (e.g., energy_efficiency.xlsx).")
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 10 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   X1      768 non-null    float64
 1   X2      768 non-null    float64
 2   X3      768 non-null    float64
 3   X4      768 non-null    float64
 4   X5      768 non-null    float64
 5   X6      768 non-null    int64
 6   X7      768 non-null    float64
 7   X8      768 non-null    int64
 8   Y1      768 non-null    float64
 9   Y2      768 non-null    float64
dtypes: float64(8), int64(2)
memory usage: 60.1 KB

--- First 5 Rows ---
     X1     X2     X3      X4   X5  X6   X7  X8     Y1     Y2
0  0.98  514.5  294.0  110.25  7.0   2  0.0   0  15.55  21.33
1  0.98  514.5  294.0  110.25  7.0   3  0.0   0  15.55  21.33
2  0.98  514.5  294.0  110.25  7.0   4  0.0   0  15.55  21.33
3  0.98  514.5  294.0  110.25  7.0   5  0.0   0  15.55  21.33
4  0.90  563.5  318.5  122.50  7.0   2  0.0   0  20.84  28.28
```

5.1 5a. Load and Pre-process Data

[27]:
```
print("Loading data and setting up features...")
df = pd.read_excel('data/energy_efficiency.xlsx')

# Separate Features (X) and Target (y)
# Drop Y1 (Heating Load) so we are predicting purely from building features␣
 ↪X1-X8
X = df.drop(columns=['Y1', 'Y2'])
y = df['Y2'] # Target: Cooling Load

# Train/Test Split (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)
```

```
# Scale the features (CRITICAL for Neural Networks to converge quickly and␣
↪properly)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Setup Early Stopping to avoid overfitting
# This monitors validation loss and stops training if it doesn't improve for 15␣
↪epochs
early_stop = EarlyStopping(monitor='val_loss', patience=15,␣
↪restore_best_weights=True)
```

Loading data and setting up features…

5.2 5b. Build and Tune ANN Architectures

[28]:
```
print("Retraining models with independent early stopping.")

# --- Architecture 1: Base Model ---
model_1 = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1, activation='linear')
])
model_1.compile(optimizer='adam', loss='mse', metrics=['mae'])
early_stop_1 = EarlyStopping(monitor='val_loss', patience=15,␣
↪restore_best_weights=True)
history_1 = model_1.fit(X_train_scaled, y_train, validation_split=0.2,
                        epochs=200, batch_size=32, callbacks=[early_stop_1],␣
↪verbose=0)

# --- Architecture 2: Wider Model with Dropout ---
model_2 = Sequential([
    Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1, activation='linear')
])
model_2.compile(optimizer='adam', loss='mse', metrics=['mae'])
early_stop_2 = EarlyStopping(monitor='val_loss', patience=15,␣
↪restore_best_weights=True)
history_2 = model_2.fit(X_train_scaled, y_train, validation_split=0.2,
                        epochs=200, batch_size=32, callbacks=[early_stop_2],␣
↪verbose=0)
```

```
# --- Architecture 3: Deeper Model ---
model_3 = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1, activation='linear')
])
model_3.compile(optimizer='adam', loss='mse', metrics=['mae'])
early_stop_3 = EarlyStopping(monitor='val_loss', patience=15,␣
 ↪restore_best_weights=True)
history_3 = model_3.fit(X_train_scaled, y_train, validation_split=0.2,
                        epochs=200, batch_size=32, callbacks=[early_stop_3],␣
 ↪verbose=0)
```

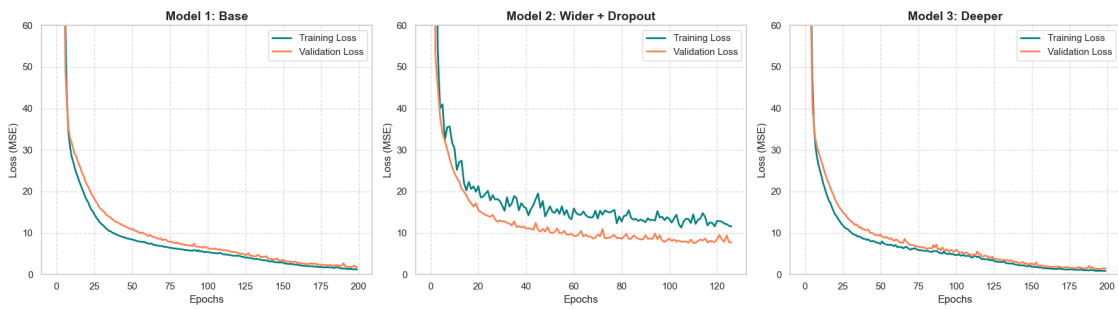
Retraining models with independent early stopping.

5.3 5c. Evaluation and Loss Plots

[29]:
```
print("\n--- Corrected Model Evaluation on Test Set ---")
models = [('Model 1 (Base)', model_1), ('Model 2 (Wider + Dropout)', model_2),␣
 ↪('Model 3 (Deeper)', model_3)]
results = []

for name, model in models:
    loss, mae = model.evaluate(X_test_scaled, y_test, verbose=0)
    results.append({'Architecture': name, 'Test MSE': round(loss, 4), 'Test␣
 ↪MAE': round(mae, 4)})

display(pd.DataFrame(results))

# Plotting Training vs Validation Loss
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
histories = [history_1, history_2, history_3]
titles = ['Model 1: Base', 'Model 2: Wider + Dropout', 'Model 3: Deeper']

for i, ax in enumerate(axes):
    ax.plot(histories[i].history['loss'], label='Training Loss', color='teal',␣
 ↪linewidth=2)
    ax.plot(histories[i].history['val_loss'], label='Validation Loss',␣
 ↪color='coral', linewidth=2)
    ax.set_title(titles[i], fontsize=14, fontweight='bold')
    ax.set_xlabel('Epochs', fontsize=12)
    ax.set_ylabel('Loss (MSE)', fontsize=12)
    ax.set_ylim(0, 60)
    ax.legend()
```

```
    ax.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

```
--- Corrected Model Evaluation on Test Set ---

                Architecture  Test MSE  Test MAE
0             Model 1 (Base)    2.1657    1.1143
1  Model 2 (Wider + Dropout)    6.7152    1.7698
2           Model 3 (Deeper)    1.8356    0.9657
```



6 Question 6

6.0.1 Data Loading (Subset)

[31]:
```
print("Loading data from text files.")

def load_imdb_data(base_dir, split, sample_size_per_class):
    texts, labels = [], []
    for label_dir, label in [('pos', 1), ('neg', 0)]:
        dir_path = os.path.join(base_dir, split, label_dir)
        files = os.listdir(dir_path)[:sample_size_per_class] # Subset
        for file in files:
            with open(os.path.join(dir_path, file), 'r', encoding='utf-8') as f:
                texts.append(f.read())
                labels.append(label)
    return texts, labels

# Load 500 pos / 500 neg for training (1000 total)
# Load 250 pos / 250 neg for testing (500 total)
train_texts, train_labels = load_imdb_data('data/aclImdb_data', 'train', 500)
test_texts, test_labels = load_imdb_data('data/aclImdb_data', 'test', 250)
```

```
print(f"Loaded {len(train_texts)} training reviews and {len(test_texts)}␣
↪testing reviews.\n")
```

```
Loading data from text files.
Loaded 1000 training reviews and 500 testing reviews.
```

6.0.2 Baseline: TF-IDF + Naive Bayes

[32]:
```
print("--- Training Baseline (TF-IDF + Naive Bayes) ---")

# Timing Training
start_train_nb = time.time()
vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = vectorizer.fit_transform(train_texts)
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, train_labels)
train_time_nb = time.time() - start_train_nb

# Timing Inference
start_inf_nb = time.time()
X_test_tfidf = vectorizer.transform(test_texts)
nb_preds = nb_model.predict(X_test_tfidf)
inf_time_nb = time.time() - start_inf_nb

# Metrics
nb_acc = accuracy_score(test_labels, nb_preds)
nb_prec = precision_score(test_labels, nb_preds, average='macro')
nb_rec = recall_score(test_labels, nb_preds, average='macro')
```

```
--- Training Baseline (TF-IDF + Naive Bayes) ---
```

6.1 6a. Transformer: DistilBERT Fine-Tuning

[33]:
```
print("--- Fine-tuning DistilBERT (This will take a few minutes!) ---")

# Load Tokenizer and encode data
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

train_encodings = tokenizer(train_texts, truncation=True, padding=True,␣
↪max_length=256)
test_encodings = tokenizer(test_texts, truncation=True, padding=True,␣
↪max_length=256)

# Create Torch Dataset class
class IMDbDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
```

```
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.
 ↪items()}
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = IMDbDataset(train_encodings, train_labels)
test_dataset = IMDbDataset(test_encodings, test_labels)

# Load pre-trained model
model = AutoModelForSequenceClassification.
 ↪from_pretrained("distilbert-base-uncased", num_labels=2)

# Set up Trainer
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=2,              # Keep low for speed
    per_device_train_batch_size=16,
    per_device_eval_batch_size=64,
    warmup_steps=50,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    report_to="none" # disable wandb/other logging
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset
)

# Timing Training
start_train_bert = time.time()
trainer.train()
train_time_bert = time.time() - start_train_bert

# Timing Inference
start_inf_bert = time.time()
bert_preds_raw = trainer.predict(test_dataset)
bert_preds = np.argmax(bert_preds_raw.predictions, axis=1)
```

```
inf_time_bert = time.time() - start_inf_bert

# Metrics
bert_acc = accuracy_score(test_labels, bert_preds)
bert_prec = precision_score(test_labels, bert_preds, average='macro')
bert_rec = recall_score(test_labels, bert_preds, average='macro')
```

```
--- Fine-tuning DistilBERT (This will take a few minutes!) ---

tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]

config.json:   0%|          | 0.00/483 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

model.safetensors:   0%|          | 0.00/268M [00:00<?, ?B/s]

Some weights of DistilBertForSequenceClassification were not initialized from
the model checkpoint at distilbert-base-uncased and are newly initialized:
['classifier.bias', 'classifier.weight', 'pre_classifier.bias',
'pre_classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it
for predictions and inference.

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>
```

6.2 6b. Comparison Results

[34]:
```
results = [
    {
        'Model': 'TF-IDF + Naive Bayes',
        'Accuracy': round(nb_acc, 4),
        'Precision': round(nb_prec, 4),
        'Recall': round(nb_rec, 4),
        'Train Time (sec)': round(train_time_nb, 2),
        'Inference Time (sec)': round(inf_time_nb, 2)
    },
    {
        'Model': 'DistilBERT',
        'Accuracy': round(bert_acc, 4),
        'Precision': round(bert_prec, 4),
        'Recall': round(bert_rec, 4),
        'Train Time (sec)': round(train_time_bert, 2),
        'Inference Time (sec)': round(inf_time_bert, 2)
    }
]
```

```
print("\n--- Model Comparison Results ---")
results_df = pd.DataFrame(results)
display(results_df)
```

```
--- Model Comparison Results ---

                  Model  Accuracy  Precision  Recall  Train Time (sec)  \
0  TF-IDF + Naive Bayes     0.798     0.8062   0.798              0.34
1            DistilBERT     0.860     0.8668   0.860           1679.83

   Inference Time (sec)
0                  0.09
1                103.76
```

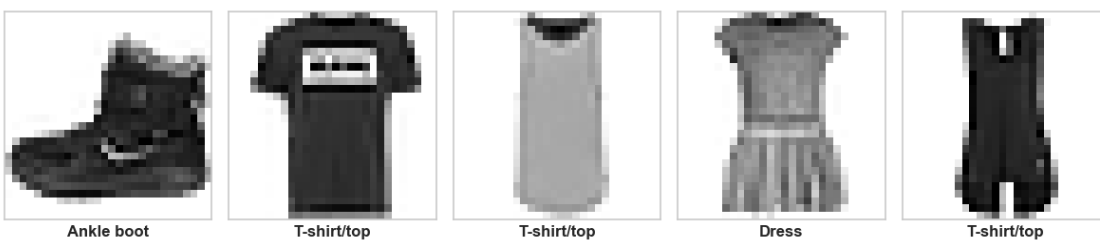
7 Question 7

7.0.1 Download and Inspect Fashion-MNIST

[35]:
```
print("Downloading and loading Fashion-MNIST dataset...")
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.
 ↪load_data()

print("\n--- Dataset Structure ---")
print(f"Training Images: {x_train.shape} | Training Labels: {y_train.shape}")
print(f"Testing Images:  {x_test.shape}  | Testing Labels:  {y_test.shape}")
print(f"Pixel Data Type: {x_train.dtype}    | Min Pixel Value: {x_train.min()}␣
 ↪| Max Pixel Value: {x_train.max()}")

# Fashion-MNIST doesn't come with string labels, so we map them manually
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

print("\n--- Visualizing First 5 Images ---")
plt.figure(figsize=(12, 3))
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    # The images are grayscale, so we use a binary color map
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]], fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()
```

```
Downloading and loading Fashion-MNIST dataset…
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-labels-idx1-ubyte.gz
29515/29515             0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/train-images-idx3-ubyte.gz
26421880/26421880             1s
0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-labels-idx1-ubyte.gz
5148/5148             0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/t10k-images-idx3-ubyte.gz
4422102/4422102             0s
0us/step

--- Dataset Structure ---
Training Images: (60000, 28, 28) | Training Labels: (60000,)
Testing Images:  (10000, 28, 28)  | Testing Labels:  (10000,)
Pixel Data Type: uint8    | Min Pixel Value: 0 | Max Pixel Value: 255

--- Visualizing First 5 Images ---
```



7.1 7a. Data Preprocessing

[37]:
```
print("Preprocessing data...")
# Normalize pixel values to be between 0 and 1
x_train_norm = x_train.astype('float32') / 255.0
x_test_norm = x_test.astype('float32') / 255.0

# Reshape to add the channel dimension (28, 28, 1) required by CNNs
x_train_cnn = np.expand_dims(x_train_norm, -1)
x_test_cnn = np.expand_dims(x_test_norm, -1)

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
Preprocessing data…
```

7.2 7a. Build CNN Architectures

[38]:
```
print("Building and training CNN models (This will take a few minutes)...\n")
epochs = 5 # Kept to 5 for reasonable local training time

# --- Architecture 1: Base CNN ---
model_base = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model_base.compile(optimizer='adam', loss='sparse_categorical_crossentropy',␣
 ↪metrics=['accuracy'])

print("Training Base CNN...")
hist_base = model_base.fit(x_train_cnn, y_train, epochs=epochs,␣
 ↪validation_data=(x_test_cnn, y_test), verbose=1)

# --- Architecture 2: CNN with Batch Normalization ---
model_bn = models.Sequential([
    layers.Conv2D(32, (3, 3), input_shape=(28, 28, 1), use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dense(10, activation='softmax')
])
model_bn.compile(optimizer='adam', loss='sparse_categorical_crossentropy',␣
 ↪metrics=['accuracy'])

print("\nTraining CNN with Batch Normalization...")
hist_bn = model_bn.fit(x_train_cnn, y_train, epochs=epochs,␣
 ↪validation_data=(x_test_cnn, y_test), verbose=1)

# --- Architecture 3: CNN with Data Augmentation ---
data_augmentation = tf.keras.Sequential([
```

```
  layers.RandomFlip("horizontal"),
  layers.RandomRotation(0.1),
  layers.RandomZoom(0.1),
])

model_aug = models.Sequential([
    data_augmentation,
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
model_aug.compile(optimizer='adam', loss='sparse_categorical_crossentropy',␣
 ↪metrics=['accuracy'])

print("\nTraining CNN with Data Augmentation...")
hist_aug = model_aug.fit(x_train_cnn, y_train, epochs=epochs,␣
 ↪validation_data=(x_test_cnn, y_test), verbose=1)
```

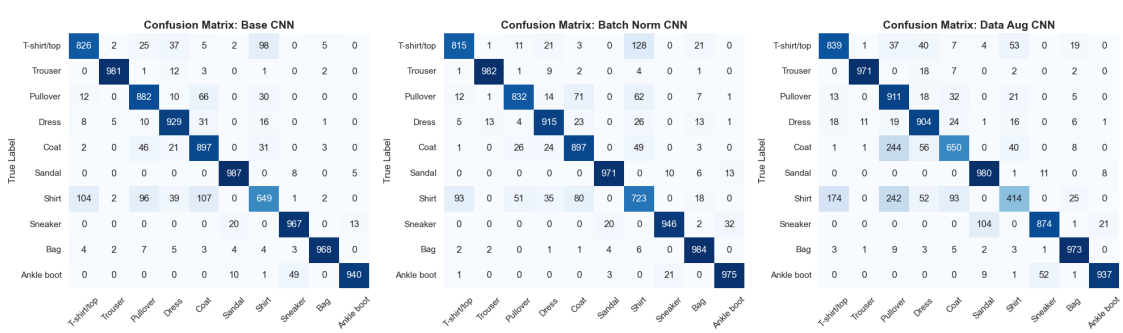
```
Building and training CNN models (This will take a few minutes)…

Training Base CNN…
Epoch 1/5
1875/1875              29s 14ms/step -
accuracy: 0.8339 - loss: 0.4611 - val_accuracy: 0.8732 - val_loss: 0.3521
Epoch 2/5
1875/1875              23s 12ms/step -
accuracy: 0.8885 - loss: 0.3099 - val_accuracy: 0.8877 - val_loss: 0.3158
Epoch 3/5
1875/1875              23s 12ms/step -
accuracy: 0.9022 - loss: 0.2646 - val_accuracy: 0.8919 - val_loss: 0.3024
Epoch 4/5
1875/1875              41s 12ms/step -
accuracy: 0.9140 - loss: 0.2327 - val_accuracy: 0.8979 - val_loss: 0.2916
Epoch 5/5
1875/1875              23s 12ms/step -
accuracy: 0.9231 - loss: 0.2072 - val_accuracy: 0.9026 - val_loss: 0.2765

Training CNN with Batch Normalization…
Epoch 1/5
1875/1875              45s 23ms/step -
accuracy: 0.8637 - loss: 0.3841 - val_accuracy: 0.8783 - val_loss: 0.3344
Epoch 2/5
1875/1875              55s 29ms/step -
accuracy: 0.9031 - loss: 0.2621 - val_accuracy: 0.8976 - val_loss: 0.2908
```

```
Epoch 3/5
1875/1875              69s 22ms/step -
accuracy: 0.9216 - loss: 0.2150 - val_accuracy: 0.8979 - val_loss: 0.2918
Epoch 4/5
1875/1875              41s 22ms/step -
accuracy: 0.9346 - loss: 0.1790 - val_accuracy: 0.8888 - val_loss: 0.3236
Epoch 5/5
1875/1875              41s 22ms/step -
accuracy: 0.9460 - loss: 0.1496 - val_accuracy: 0.9040 - val_loss: 0.2996

Training CNN with Data Augmentation…
Epoch 1/5
1875/1875              29s 14ms/step -
accuracy: 0.7449 - loss: 0.6946 - val_accuracy: 0.7887 - val_loss: 0.6104
Epoch 2/5
1875/1875              27s 14ms/step -
accuracy: 0.8135 - loss: 0.5059 - val_accuracy: 0.8251 - val_loss: 0.5066
Epoch 3/5
1875/1875              28s 15ms/step -
accuracy: 0.8357 - loss: 0.4473 - val_accuracy: 0.8310 - val_loss: 0.4828
Epoch 4/5
1875/1875              25s 14ms/step -
accuracy: 0.8507 - loss: 0.4076 - val_accuracy: 0.8381 - val_loss: 0.4577
Epoch 5/5
1875/1875              27s 15ms/step -
accuracy: 0.8583 - loss: 0.3824 - val_accuracy: 0.8453 - val_loss: 0.4457
```

7.3 7b. Evaluation and Confusion Matrices

[39]:
```python
print("\n--- Model Evaluation ---")
results = []
histories = [hist_base, hist_bn, hist_aug]
model_names = ['Base CNN', 'Batch Norm CNN', 'Data Aug CNN']
models_list = [model_base, model_bn, model_aug]

for i in range(3):
    train_loss = histories[i].history['loss'][-1]
    train_acc = histories[i].history['accuracy'][-1]
    test_loss = histories[i].history['val_loss'][-1]
    test_acc = histories[i].history['val_accuracy'][-1]

    results.append({
        'Architecture': model_names[i],
        'Train Loss': round(train_loss, 4),
        'Train Accuracy': round(train_acc, 4),
        'Test Loss': round(test_loss, 4),
        'Test Accuracy': round(test_acc, 4)
```

```
    })

display(pd.DataFrame(results))

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for i, model in enumerate(models_list):
    # Get predictions
    y_pred_probs = model.predict(x_test_cnn, verbose=0)
    y_pred = np.argmax(y_pred_probs, axis=1)

    # Generate matrix
    cm = confusion_matrix(y_test, y_pred)

    # Plot
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i],
                xticklabels=class_names, yticklabels=class_names, cbar=False)
    axes[i].set_title(f'Confusion Matrix: {model_names[i]}', fontsize=14,␣
 ↪fontweight='bold')
    axes[i].set_xlabel('Predicted Label')
    axes[i].set_ylabel('True Label')
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

```
--- Model Evaluation ---
```

	Architecture	Train Loss	Train Accuracy	Test Loss	Test Accuracy
0	Base CNN	0.2072	0.9231	0.2765	0.9026
1	Batch Norm CNN	0.1496	0.9460	0.2996	0.9040
2	Data Aug CNN	0.3824	0.8583	0.4457	0.8453